Factory Method

ry Method · Create Email/SMS/Push notifiers without client coupling · Build with W

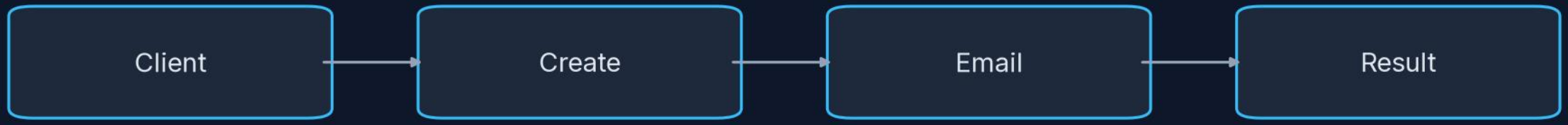
Today's Hook

- Problem: Create Email/SMS/Push notifiers without client coupling
- Pattern: Factory Method — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Create Email/SMS/Push notifiers without client coupling
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Factory Method: Create Email/SMS/Push notifiers without client cou

Real-World Scenario

- Running example: Create Email/SMS/Push notifiers without client coupling
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.creational.factorymethod;

import com.buildwithwaqar.patterns.gof.creational.factorymethod.*;

/** Problem: notify users on signup without hard-coding Email/SMS/Push in controllers. */
public final class UseCaseDemo {
    public static void main(String[] args) {
        System.out.println("=== Factory Method problem: channel-agnostic notify ===");
        System.out.println("naive would: new EmailNotification().deliver(...) everywhere");
        for (var ch : NotificationChannel.values()) {
            System.out.println(ch + " -> " + ch.notify("user@x", "Welcome to Build with Waqar"));
        }
    }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.creational.factorymethod;

/** Modern Java 25: sealed enum + switch expression replaces the class hierarchy. */
public enum NotificationChannel {
    EMAIL, SMS, PUSH;

    public Notification create() {
        return switch (this) {
            case EMAIL -> new EmailNotification();
            case SMS -> new SmsNotification();
            case PUSH -> new PushNotification();
        };
    }

    public String notify(String to, String body) {
        return create().deliver(to, body);
    }
}


package com.buildwithwaqar.patterns.gof.creational.factorymethod;
```

Solution (2)

```java
/** Creator hierarchy — subclasses decide which product to instantiate. */
public abstract class Notifier {
    protected abstract Notification create();

    public final String notify(String to, String body) {
        return create().deliver(to, body);
    }
}

final class EmailNotifier extends Notifier {
    @Override protected Notification create() { return new EmailNotification(); }
}

final class SmsNotifier extends Notifier {
    @Override protected Notification create() { return new SmsNotification(); }
}

final class PushNotifier extends Notifier {
    @Override protected Notification create() { return new PushNotification(); }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.creational.factorymethod;

import com.buildwithwaqar.patterns.gof.creational.factorymethod.*;

/** Problem: notify users on signup without hard-coding Email/SMS/Push in controllers. */
public final class UseCaseDemo {
    public static void main(String[] args) {
        System.out.println("=== Factory Method problem: channel-agnostic notify ===");
        System.out.println("naive would: new EmailNotification().deliver(...) everywhere");
        for (var ch : NotificationChannel.values()) {
            System.out.println(ch + " -> " + ch.notify("user@x", "Welcome to Build with Waqar"));
        }
    }
}
```

Walkthrough Output

```
=== Factory Method problem: channel-agnostic notify ===
naive would: new EmailNotification().deliver(...) everywhere
EMAIL -> email -> user@x: Welcome to Build with Waqar
SMS -> sms -> user@x: Welcome to Build with Waqar
PUSH -> push -> user@x: Welcome to Build with Waqar
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Factory Method.

Related Patterns

- Contrast Factory Method with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Factory Method → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.